Design Pattern Analysis in a Hospital Management System Using Microservices and Spring Boot

Team A16

Sai Krishna — AM.EN.U4EAC23033
Siddhartha — AM.EN.U4EAC23042
Sai Rudra — AM.EN.U4EAC23048
Shradhey K — AM.EN.U4EAC23064
Pavan Sai — AM.EN.U4EAC23025

2026-10-29

Contents

1 Application and Architecture **2**

2 Part 1 Pattern Discovery **2**
2.1 Totals at the baseline . . . 2
2.2 Pattern inventory . . . 3
2.3 One record for each pattern instance . . . 3
2.4 Qualified, rejected, and uncertain candidates . . . 4
2.5 Baseline risks and maintainability observations . . . 4

3 Part 2 Evolution and Refactoring **5**
3.1 CH01 — Service Layer extraction . . . 5
3.2 CH02 — Strategy for patient-validation transports . . . 5
3.3 CH03 — Observer / Spring application event . . . 5
3.4 Updated pattern counts . . . 6

4 Part 3 Cross-Language Comparison **6**
4.1 Actual results and comparison . . . 7

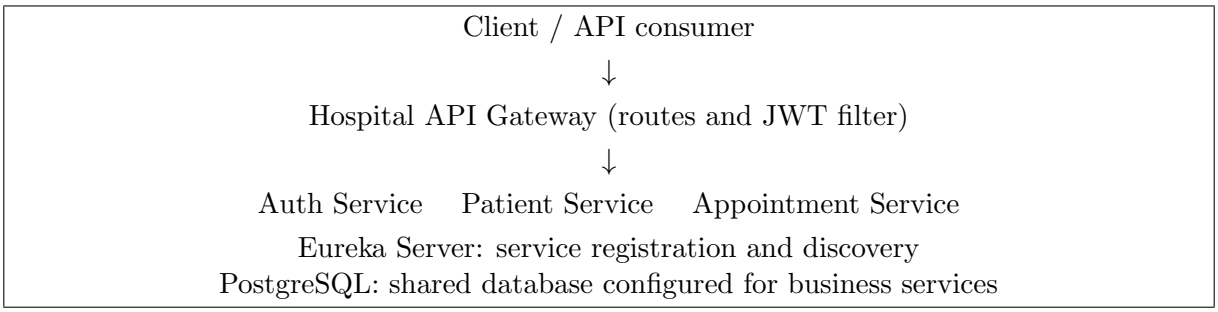
5 LLM Records and Human Verification **7**

6 Findings, Contributions and References **7**

7 Team Folder and Datasets **8**

1 Application and Architecture

Original source. Repository URL above; analysis version is the ZIP snapshot supplied for this report on 2026-10-29. No immutable Git commit hash or release tag was supplied, so the snapshot cannot be identified by commit. The repository's declared license was not verified in the supplied files.
Environment. Java 17 as stated by the README; Spring Boot, Spring Cloud Gateway, Spring Data JPA, Spring Security/JWT, Maven, PostgreSQL, and Netflix Eureka. Exact resolved dependency versions should be taken from the Maven descriptors for a reproducible build.
Scope. Inspected README, Java source, Maven descriptors, application properties, repository/entity/controller classes, and service entry points across five modules: `EurekaServer`, `hospital-gateway`, `auth-service`, `patient-service`, and `appointment-service`. Part 2 is a separate proposed refactoring overlay; Part 3 examples are independent educational snippets.
Architecture. The system has five Spring Boot applications. The gateway routes requests to auth, patient, and appointment services. Eureka supports discovery; the appointment service calls the patient service. The README/configuration indicates that the three business services use the same PostgreSQL database, `mydb`, which creates database coupling despite separate applications.



Run instructions. The original README contains setup information. This case study did not run the entire system; therefore no end-to-end success is claimed. The commands in the package README reproduce the isolated Part 3 examples and structural tests. The Spring services should be started only after configuring database credentials, JWT secrets, and service discovery endpoints.

2 Part 1 Pattern Discovery

Method. Candidate patterns were identified by reviewing the source modules, class relationships, annotations, routes, repositories, and application properties. Findings were checked against concrete participants and interactions. Framework features were not counted as custom GoF implementations merely because their names resemble patterns.

2.1 Totals at the baseline

- Confirmed pattern types: 6 (Microservices Architecture, API Gateway, Service Discovery, Layered Architecture, Repository, Client–Server).
- Confirmed pattern instances: 8 (one each for the first four listed types, three Repository collaborations, and one Client–Server collaboration).
- Concrete Java class/interface participants mapped in the inventory: 15; architectural roles also include configuration and module symbols.
- Unique participant symbols across the records: 17, with repeated symbols counted once; architecture/module participants are not all Java classes.
- Rejected or uncertain candidates: 4 (Strategy, Observer, Decorator, Circuit Breaker). Two qualified framework observations—Dependency Injection and singleton bean scope—are documented but excluded from the six-type count.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Participant classes / symbols
Microservices Architecture	Hospital service decomposition	1	5 service/application modules
API Gateway	Inbound request routing and JWT filter	1	Gateway application, route config, filter
Service Discovery	Eureka registration and load-balanced routing	1	Eureka app, service apps, route config
Layered Architecture	Patient resource handling	1	PatientController, PatientRepo, Patient
Repository	Patient persistence; appointment persistence; user persistence	3	PatientRepo, AppointmentRepo, UserRepo and collaborating symbols
Client–Server	REST requests through gateway and service endpoints	1	API consumer, gateway, REST controllers
Total	**6 distinct types**	**8**	**See scope note above**

2.3 One record for each pattern instance

1. Instance: Hospital service decomposition.
Type/category: Microservices Architecture; architectural style.
Purpose/origin: Separate deployable applications for authentication, patients, appointments, gateway routing, and service registry.
Participant mapping: `AuthServiceApplication`; `PatientServiceApplication`; `AppointmentServiceApplication`; `HospitalGatewayApplication`; `EurekaServerApplication`. Each is in its module's Java source tree.
Evidence/verification: Independent Spring Boot entry points and module configuration support the service decomposition. Qualification: the business services share a PostgreSQL database, so independent data ownership is incomplete. Confirmed as an architectural style, not a GoF pattern.

2. Instance: Inbound request routing and JWT filter.
Type/category: API Gateway; distributed-systems pattern.
Purpose/origin: Provide a common inbound routing layer and apply a JWT-related filter.
Participant mapping: gateway application; `hospital-gateway/src/main/resources/application.properties`; `hospital-gateway/src/main/java/com/infosys/gateway/Filter/JwtAuthenticationFilter.java`.
Evidence/verification: Route configuration maps paths to auth, patient, and appointment service IDs; the filter implements Spring Cloud Gateway's global-filter contract. Confirmed from configuration and source; live gateway behavior was not run.

3. Instance: Eureka registration and load-balanced routing.
Type/category: Service Discovery; distributed-systems pattern.
Purpose/origin: Register service instances and allow service-ID-based routing.
Participant mapping: `EurekaServer/.../EurekaServerApplication.java`; service application classes with discovery configuration; gateway routes using `lb://` service IDs; load-balanced HTTP client configuration in appointment service.
Evidence/verification: Eureka server annotation and discovery configuration are present. Confirmed structurally; registration and routing were not verified at runtime.

4. Instance: Patient resource handling.
Type/category: Layered Architecture; architectural pattern.
Purpose/origin: Separate HTTP request handling, persistence abstraction, and data representation.
Participant mapping: `patient-service/.../Controller/PatientController.java`; `patient-service/.../Repository/PatientRepo.java`; `patient-service/.../Entity/Patient.java`.
Evidence/verification: These roles reside in separate packages and collaborate through the con-

troller/repository/entity structure. The separation is partial because business and persistence concerns remain in controllers. Confirmed with caveat.

5. Instance: Patient persistence.
Type/category: Repository; data-access pattern/framework abstraction.
Purpose/origin: Provide a persistence interface for patient records.
Participant mapping: `PatientRepo` extends Spring Data JPA `JpaRepository`; collaborating model: `Patient`; consumer: `PatientController`.
Evidence/verification: Repository interface delegates standard persistence behavior to Spring Data JPA. Confirmed as a framework-backed Repository abstraction.

6. Instance: Appointment persistence.
Type/category: Repository; data-access pattern/framework abstraction.
Participant mapping: `AppointmentRepo`; appointment entity; consumer: `AppointmentController` in the appointment service.
Evidence/verification: The repository extends `JpaRepository` and is used to persist/retrieve appointment records. Confirmed as a framework-backed Repository abstraction.

7. Instance: User persistence.
Type/category: Repository; data-access pattern/framework abstraction.
Participant mapping: `UserRepo`; user entity/model; consumer: auth controller/service code.
Evidence/verification: The repository extends `JpaRepository` and abstracts user persistence operations. Confirmed as a framework-backed Repository abstraction.

8. Instance: REST requests through gateway and service endpoints.
Type/category: Client–Server; architectural style.
Participant mapping: API consumer (e.g. Postman or another HTTP client); gateway route configuration; REST controllers in auth, patient, and appointment services.
Evidence/verification: Controllers expose HTTP operations and the gateway/service clients communicate through HTTP. Confirmed from endpoints and routing configuration; end-to-end calls were not run.

2.4 Qualified, rejected, and uncertain candidates

- **Dependency Injection / IoC — qualified, not included in the six pattern types.** Spring injects repositories, JWT services, encoders, and HTTP clients. This is an IoC principle/framework mechanism, not a custom GoF implementation. Field injection appears in the baseline.
- **Singleton bean scope — qualified, not included in the six pattern types.** Spring components are singleton-scoped by default unless configured otherwise. This does not establish a hand-written Singleton pattern.
- **Strategy — rejected for baseline.** RestTemplate and WebClient paths are separate methods; no shared strategy contract or polymorphic selection is present.
- **Observer — rejected for baseline.** No application-defined subject/subscriber collaboration or event-listener relationship was identified in the inspected Java source.
- **Decorator — not established.** Annotations and a gateway filter alone do not demonstrate a custom Decorator implementation.
- **Circuit Breaker — not found.** No explicit circuit-breaker implementation/configuration was established in the inspected source.

2.5 Baseline risks and maintainability observations

1. `AppointmentController` combines request handling, patient validation, downstream calls, status defaults, and persistence; appointment creation paths duplicate workflow logic.
2. Missing records are sometimes represented by `null`, and delete operations use weak result semantics; consistent HTTP error mapping is needed.
3. Database credentials and a JWT signing secret are hard-coded in source/configuration. Rotate any real exposed secrets and externalize them before deployment.
4. Gateway configuration exposes all Actuator endpoints with a wildcard setting; restrict and protect management endpoints.

5. A shared PostgreSQL database creates coupling between otherwise separately deployed business services.
6. Tracing dependencies/configuration do not prove live Zipkin collection or end-to-end trace propagation; runtime verification remains outstanding.

3 Part 2 Evolution and Refactoring

The baseline snapshot remains in `Part1_Baseline/original_project`. Refactoring files are stored separately in `Part2_Refactoring/refactored_project` and the overlay/diff folders. These are proposed changes, not a verified production release.

3.1 CH01 — Service Layer extraction

Change and acceptance criteria. Move appointment use-case orchestration out of the controller. A successful create request should still validate the patient, apply the default status, persist the appointment, and return the saved result.
Before/after paths. Before: `appointment-service/.../Controller/AppointmentController.java`. After: controller delegates to `AppointmentService`; service owns validation/persistence workflow.
Implementation and tests. The structural test `test_service_layer_exists` and `test_controller_delegates_to_service` passed. The Spring overlay was not compiled or integration-tested.
Pattern effects. Service Layer is newly introduced; existing API Gateway, Service Discovery, and Repository collaborations are intended to remain.
Refactoring and metrics. Extract Method / Move Method style separation addresses a controller with multiple responsibilities and duplicated use-case flow. LOC metrics are not reported as a quality proxy; no behavior-preservation integration test has been run.

3.2 CH02 — Strategy for patient-validation transports

Change and acceptance criteria. Provide one `PatientValidationStrategy` contract with RestTemplate and WebClient implementations, while preserving the existing endpoint behavior.
Before/after paths. Before: duplicated validation logic in appointment controller methods. After: `PatientValidationStrategy`, `RestTemplatePatientValidationStrategy`, and `WebClientPatientValidationStrategy` in the refactored appointment-service source.
Implementation and tests. Structural check `test_strategy_contract_and_two_implementations_exist` passed. Runtime downstream behavior and HTTP error handling remain unverified.
Pattern effects. Strategy is newly introduced in CH02; it is not a baseline discovery.
Refactoring and metrics. Extracting transport-specific validation reduces duplication and isolates variation. Two strategies are justified only while both transports are supported; otherwise one transport may be simpler.

3.3 CH03 — Observer / Spring application event

Change and acceptance criteria. After successful persistence, publish an appointment-created event and let an independent listener perform audit logging. The main appointment use case should not call the audit component directly.
Before/after paths. Before: no application-defined appointment-created event listener in the inspected baseline. After: `AppointmentCreatedEvent`, publication in `AppointmentService`, and `AppointmentAuditObserver` with Spring `@EventListener`.
Implementation and tests. Structural checks `test_observer_event_and_listener_exist` and controller-delegation checks passed. Event dispatch was not integration-tested in a running Spring context.
Pattern effects. Observer is newly introduced in CH03. It is an in-process event, not durable messaging and not a guarantee of external notification delivery.

Refactoring and metrics. Separating audit logging reduces coupling. A transactional event/outbox or message broker would be needed for stronger delivery guarantees.

3.4 Updated pattern counts

Snapshot	Distinct types	Instances	Participant symbols	Interpretation
Baseline	6	8	17	Original source findings only.
After CH01	7	9	18	Adds Service Layer collaboration; baseline patterns retained.
After CH02	8	10	21	Adds Strategy interface and two implementations.
After CH03	9	11	23	Adds event and observer participants.

Counts are inventory-level counts for the named collaborations, not a universal measure of design quality. Shared participants are counted once within the unique-symbol totals. Architectural styles have module/configuration participants that are not all Java classes. The structural test suite contains five checks and all five passed; these checks do not replace Maven compilation, database integration, or end-to-end tests.

Before/after test evidence. See `verification/refactoring_structural_tests.txt` and `evidence/verification_summary.txt`. Full integration is not verified.

Remaining limitations. Add tests for missing patients, downstream 4xx/5xx and timeout responses, persistence failure, duplicate requests, event publication, and consistent exception-to-HTTP mapping. Avoid exposing raw exception messages.

4 Part 3 Cross-Language Comparison

Selected instances. Repository (in-memory keyed store), Strategy (standard and senior fee calculation), and Observer (appointment-created callback) were selected because they can be represented consistently across the four required languages. These are educational analogues, not production replacements for Spring Data JPA or a durable event system. The senior fee reduction is a test fixture, not a real hospital billing policy.

Common specification. Repository: save, find, missing-key behavior, and delete. Strategy: return expected fee for standard and senior policies. Observer: registered callback receives the created appointment identifier. The same core cases are represented in Java, Python, JavaScript, and C++.

Implementations and commands.

- Java: `Part3_CrossLanguage/Java/Patterns.java`; run `javacPatterns.java&&javaPatterns`.
- Python: `Part3_CrossLanguage/Python/patterns.py`; run `python3-munittest-v` from the Python folder.
- JavaScript: `Part3_CrossLanguage/JavaScript/patterns.test.js`; run `node--testpatterns.test.js`.
- C++: `Part3_CrossLanguage/Cpp/patterns_test.cpp`; run `g++-std=c++17-O2patterns_test.cpp-opatterns_test&&./patterns_test`.

4.1 Actual results and comparison

Language	Result	Pattern coverage	Evidence
Python	6 tests passed	Repository, Strategy, Observer	unittest output in evidence log.
Java	Passed	Repository, Strategy, Observer	javac compilation and runtime assertions.
JavaScript	Passed	Repository, Strategy, Observer	Node test output.
C++	Passed	Repository, Strategy, Observer	g++ C++17 compilation and runtime assertions.

The test matrix contains 12 pattern/language combinations, all recorded as passing. Java interfaces and classes make contracts explicit but require more boilerplate; Python can express the same roles concisely; JavaScript functions/classes support callback-based observers; C++ offers interfaces through abstract classes and templates/callables but requires careful lifetime management. These are qualitative observations. No performance benchmark or language speed ranking was run.

5 LLM Records and Human Verification

The prompt summaries are stored in `prompts/part1_baseline_analysis_prompt.md`, `prompts/part2_refactoring_prompt.md`, and `prompts/part3_cross_language_prompt.md`. These are reconstructed summaries of the task, not exported raw provider logs. `data/llm_records.csv` records the stage, inputs, outputs, verification method, and limitations. Human verification consisted of source-path review, inspection of the refactoring diff, five structural tests, and the recorded cross-language tests. The full Spring Boot system was not launched; all such runtime claims remain unverified.

6 Findings, Contributions and References

Findings. The strongest baseline patterns are Microservices Architecture, API Gateway, Service Discovery, Layered Architecture (partially separated), Repository, and Client–Server. Dependency Injection and singleton bean scope are framework conventions rather than custom GoF implementations. The most defensible evolution is Service Layer extraction, Strategy for two patient-validation transports, and an Observer-based audit event. Shared database ownership, hard-coded secrets, broad Actuator exposure, and incomplete runtime verification remain limitations.

Contributions. Team roster is confirmed; person-specific allocations were not provided. The team must replace the following shared-work placeholder with actual responsibilities before submission. No individual tasks are attributed without evidence.

Roll number	Name	Actual contribution (team to complete)
AM.EN.U4EAC23048	Sai Rudra	found the project repo.
AM.EN.U4EAC23033	Sai Krishna	found the initial design pattrens.
AM.EN.U4EAC23025	Pavan Sai	found the initial design pattrens.
AM.EN.U4EAC23042	Siddhartha	modified and edited the founded pattrens.
AM.EN.U4EAC23064	Shradhey K	modified and edited the founded pattrens.

References.

1. Risshhhiiii, *Hospital Management System Using MicroServices And SpringBoot*, supplied repository snapshot and README, `https://github.com/Risshhhiiii/Hospital-Management-System-Using-MicroServices-And-SpringBoot`.
2. Gamma, E., Helm, R., Johnson, R., and Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
3. Spring Boot Reference Documentation: `https://docs.spring.io/spring-boot/`.

4. Spring Cloud Gateway Reference: https://docs.spring.io/spring-cloud-gateway/reference/.
5. Spring Cloud Netflix / Eureka documentation: https://docs.spring.io/spring-cloud-netflix/reference/.
6. Spring Data JPA Reference Documentation: https://docs.spring.io/spring-data/jpa/reference/.

7 Team Folder and Datasets

The submission folder follows the provided report template's requested organization, with A16-specific names:

```
A16/
  README.md
  A16_Report.tex
  A16_Report.pdf
  A16_Report.docx
  TEAM_A16.md
  CONTRIBUTIONS.md
  data/ pattern_inventory.csv, source_file_inventory.csv,
        cross_language_test_matrix.csv, llm_records.csv
  Part1_Baseline/ original_project/, baseline_analysis.md
  Part2_Refactoring/ refactored_project/, diffs/, overlay/
  Part3_CrossLanguage/ Java/, Python/, JavaScript/, Cpp/
  prompts/ reconstructed prompt summaries
  tests/ structural tests
  evidence/ and verification/ recorded test/build results
```

Screenshots of the final GitHub repository should be captured by the team after uploading the folder; no screenshot is fabricated in this report.